

Controle Fuzzy Mamdani Otimizado por Algoritmo Genético para Rastreamento de Trajetória de Robô Autônomo: uma Reprodução de Mancilla et al. (2022)

Disciplina: Inteligência Artificial e Computacional (0700M8) — CESUPA **Turma:** CC5MA **Equipe (5 integrantes):** Brenda Nascimento, Cauê Jadão, Augusto Pereira, Fernando Mourão, César Ribeiro **Modalidade:** Opção A — Pesquisa em artigos científicos, trilha **A1 (Reprodução)**, com extensão opcional de otimização por **Algoritmo Genético (AG)** (pontuação extra, opção 2) **Trilha de ampliação obrigatória (equipe de 5): Comparação de modelos** — duas configurações de funções de pertinência comparadas (baseline × otimizada pelo AG); ver Seção “Equipe de 5 integrantes” **Repositório GitHub:** <https://github.com/brenda24070071-dotcom/fuzzy-ag-path-tracking-cesupa>

Resumo

Este trabalho reproduz, de forma mínima viável, o sistema de controle fuzzy proposto por Mancilla, García-Valdez, Castillo e Merelo-Guervós (*Symmetry*, 2022) para rastreamento de trajetória de um robô autônomo com cinemática de bicicleta. O controlador é um sistema **Mamdani** com duas entradas — erro lateral (m) e erro angular (rad) em relação à trajetória de referência — e uma saída — taxa de guinada (rad/s) —, cada variável com cinco termos linguísticos e base de **25 regras**. As funções de pertinência das entradas são parametrizadas por um vetor de 10 genes e otimizadas por um **algoritmo genético real-coded** (seleção por torneio, cruzamento aritmético, mutação gaussiana e elitismo), com função objetivo igual ao RMSE médio do erro lateral em três pistas de referência interpoladas por spline cúbica. Em cinco execuções independentes com sementes fixas, a otimização reduziu o RMSE médio de **1,560 m (baseline)** para **1,232 m**, uma melhoria de **21,0%**, com o robô completando as três pistas em todos os casos. Documentamos ainda, por honestidade metodológica, um defeito de convenção de sinais encontrado e corrigido durante a validação — evidência da indispensabilidade da validação por simulação em sistemas fuzzy.

Palavras-chave: lógica fuzzy; controle Mamdani; rastreamento de trajetória; robô móvel; algoritmo genético; otimização de funções de pertinência.

1. Introdução

Rastreamento de trajetória (*path tracking*) é o problema de comandar um veículo para seguir um caminho de referência com o menor desvio possível. É um componente central de robôs móveis, veículos autônomos e máquinas agrícolas, e um problema naturalmente adequado à lógica fuzzy: o conhecimento de um “motorista” é qualitativo (“se estou um pouco à direita e apontando para fora, vire mais forte à esquerda”), as fronteiras entre situações são graduais, e o acoplamento entre erro de

posição e erro de orientação torna soluções analíticas exatas dependentes de modelos precisos que nem sempre estão disponíveis.

Controladores fuzzy do tipo **Mamdani** [3] permitem codificar esse conhecimento linguístico diretamente em regras interpretáveis. Entretanto, o desempenho final depende fortemente da forma e posição das **funções de pertinência (MFs)** — ajuste tradicionalmente manual e trabalhoso. O artigo reproduzido neste trabalho, Mancilla et al. (2022) [1], ataca esse ponto: usa **metaheurísticas populacionais** para otimizar automaticamente as MFs de um controlador Mamdani de rastreamento, avaliando cada candidato por simulação do robô em pistas de referência.

Objetivos deste trabalho: (i) reproduzir a estrutura do controlador e o protocolo de avaliação do artigo (reprodução mínima viável — trilha A1); (ii) implementar a otimização das MFs por AG, conectando o trabalho à computação evolutiva (extensão de pontuação extra); (iii) validar o sistema com cenários pontuais e simulação completa, analisando criticamente onde funciona bem e onde falha.

Por que fuzzy é adequado aqui: o problema envolve controle aproximado com julgamento linguístico, gradação contínua entre situações (não há fronteira nítida entre “pouco” e “muito” desviado) e tolerância a imprecisão de modelo — exatamente o perfil de aplicação para o qual a inferência fuzzy foi concebida [3], [6], [7].

2. Fundamentação Teórica

Conjuntos fuzzy e variáveis linguísticas. Um conjunto fuzzy A sobre um universo U é definido por uma função de pertinência $\mu_A: U \rightarrow [0,1]$ [7]. Variáveis linguísticas assumem termos (“Alto Negativo”, “Zero”, ...) modelados como conjuntos fuzzy sobre o universo de discurso da variável.

Inferência Mamdani. No controlador Mamdani [3], cada regra “SE x é A E y é B ENTÃO z é C ” é avaliada em quatro etapas: (i) **fuzzificação** das entradas (aqui, singleton); (ii) grau de ativação da regra pela t-norma **mínimo** (operador E); (iii) **implicação** por mínimo, truncando o conjunto de saída C ; (iv) **agregação** das saídas de todas as regras pelo **máximo**. A saída numérica é obtida por **defuzzificação pelo centroide**: $\omega^* = \int \omega \cdot \mu(\omega) d\omega / \int \mu(\omega) d\omega$.

Sistemas TSK (comparação conceitual). Em Takagi–Sugeno–Kang [4], os consequentes são funções (constantes ou afins) das entradas e a saída é a média ponderada dos consequentes pelos graus de ativação. TSK produz saídas mais suaves/contínuas e é mais fácil de ajustar analiticamente; Mamdani preserva interpretabilidade linguística também na saída — motivo pelo qual o artigo de referência (e esta reprodução) o adota.

Sistemas genético-fuzzy. O ajuste de MFs por algoritmos evolutivos é uma linha consolidada (*genetic fuzzy systems* [5]): o AG explora o espaço de parâmetros das MFs guiado por uma função de aptidão baseada em desempenho simulado, sem exigir gradientes.

3. Trabalhos Relacionados

O levantamento bibliográfico completo — bases de busca, palavras-chave, critérios de inclusão/exclusão e tabela com 7 trabalhos — está em `docs/trabalhos_relacionados.md`. Em síntese: [1] é o artigo reproduzido; [2] estabelece o uso de erros geométricos (lateral/angular) como entradas-padrão de controladores fuzzy de path tracking; [3] e [4] fundamentam Mamdani e TSK; [5] enquadra a otimização evolutiva de sistemas fuzzy; [6] evidencia a maturidade industrial do controle fuzzy; [7] é a base teórica de conjuntos fuzzy. A escolha de [1] como artigo principal se deve à sua reprodutibilidade (modelo, pistas, controlador e função objetivo descritos), atualidade (2022, acesso aberto) e aderência simultânea a controle fuzzy e computação evolutiva.

4. Metodologia

4.1 Modelo cinemático do robô

Modelo de **bicicleta** com distância entre eixos $L = 2,5$ m e velocidade constante $v = 10/3 \approx 3,33$ m/s, integrado por Euler com passo $\Delta t = 0,1$ s:

$$\begin{aligned}x(k+1) &= x(k) + v \cdot \cos(\theta) \cdot \Delta t \\y(k+1) &= y(k) + v \cdot \sin(\theta) \cdot \Delta t \\ \theta(k+1) &= \theta(k) + (v/L) \cdot \tan(\delta) \cdot \Delta t, \quad \delta = \arctan(L \cdot \omega / v), \quad |\delta| \leq 45^\circ\end{aligned}$$

A saída fuzzy ω (taxa de guinada) é convertida em ângulo de esterçamento δ com saturação.

4.2 Pistas de referência e sinais de erro

Três pistas (M, A e S) são geradas por **spline cúbica** sobre pontos de controle (conforme o artigo de referência), amostradas em 500 pontos. A cada passo, o ponto mais próximo da pista é obtido por busca em árvore k-d, e calculam-se:

- **erro lateral** `e_lat` : componente perpendicular do desvio em relação à tangente da pista (produto vetorial), com sinal — positivo à direita do sentido de percurso;
- **erro angular** `theta_e =  $\theta_{\text{robô}} - \theta_{\text{tangente}}$` , normalizado para $[-\pi, \pi]$.

4.3 Função objetivo e critério de término

Cada controlador candidato é avaliado pelo **RMSE do erro lateral** médio nas 3 pistas (equivalente à Eq. 12 do artigo). A simulação termina quando o robô alcança o fim da pista (ponto mais próximo entre os 3 últimos da amostragem e distância < 2 m do ponto final) ou quando estoura $T_{\text{MAX}} = 50$ s; controladores que não completam o percurso recebem penalidade de +2000, e falhas de inferência recebem 5000.

4.4 Algoritmo Genético

Componente	Escolha
Representação	Vetor real $p \in [0,1]^{10}$ (5 parâmetros de MF por entrada)
População / gerações	20 / 10 (orçamento reduzido — limitação declarada)
Seleção	Torneio de 3
Cruzamento	Aritmético (α aleatório por gene), taxa 0,7
Mutação	Gaussiana ($\sigma = 0,2$), taxa 0,3 por gene, com truncamento em $[0,1]$
Elitismo	2 melhores indivíduos
Critério de parada	Número fixo de gerações
Robustez	5 execuções independentes (sementes 42–46)

5. Modelagem Fuzzy

5.1 Variáveis e universos de discurso

Variável	Papel	Unidade	Universo	Termos
e_lat	entrada	m	$[-50, 50]$ (região fina: $[-5, 5]$)	AN, MN, Z, MP, AP
theta_e	entrada	rad	$[-50, 50]$ (operação: $[-\pi, \pi]$)	AN, MN, Z, MP, AP
omega	saída	rad/s	$[-50, 50]$	AN, MN, Z, MP, AP

5.2 Funções de pertinência

Triangulares no centro e trapezoidais nas extremidades (ombros longos garantem cobertura total do universo). As MFs das **entradas** são parametrizadas pelos 10 genes do AG; as da **saída são fixas**, preservando o significado físico da ação de controle e reduzindo o espaço de busca. Fórmulas, tabelas de parâmetros (baseline e otimizados) e a análise da mudança produzida pelo AG estão em `docs/funcoes_de_pertinencia.md`; os gráficos, em `resultados/mfs_otimizadas.png`.

5.3 Base de regras

Base completa de **25 regras** (matriz 5×5 antissimétrica) com convenções de sinal, listagem integral e justificativa por grupos (regime nominal, correção de rumo, retorno à pista, zona morta, convergência, correção composta e conflito) em `docs/base_de_regras.md`.

5.4 Inferência e defuzzificação

Mamdani **min–max**: E = mínimo; implicação = mínimo; agregação = máximo; defuzzificação = **centroide**. A implementação rápida discretiza o universo de saída em passos de 1,0 e calcula o centroide do conjunto agregado; a implementação `scikit-fuzzy` (usada para gráficos e cenários) usa passo 0,1.

6. Implementação

Arquitetura (arquivo único `fuzzy_path_tracking.py`, espelhado no notebook):

1. `Track` — geração das pistas por spline cúbica + busca de ponto mais próximo (k-d tree);
2. `FastFuzzyController` — motor Mamdani em NumPy puro (~1500× mais rápido que o `scikit-fuzzy` para este caso), usado dentro do laço do AG;
3. `build_fuzzy` — o mesmo sistema construído em `scikit-fuzzy`, usado para os cenários pontuais, superfície de controle e gráficos de MFs (validação cruzada das duas implementações);
4. `simulate` / `fitness` — simulação cinemática e função objetivo;
5. `run_ga` — algoritmo genético;
6. `main` — pipeline completo: baseline → 5 execuções do AG → comparação e gráficos.

Dependências: Python ≥ 3.10, NumPy, SciPy, Matplotlib, scikit-fuzzy (versões em `requirements.txt`). **Execução:** `python fuzzy_path_tracking.py` (~1–2 min) — manual completo em `docs/manual_de_execucao.md`.

7. Experimentos e Resultados

7.1 Cenários pontuais de inferência (6 casos)

Tabela completa com interpretação e análise de coerência em `docs/cenarios_de_teste.md`. Síntese (valores reais; ω em rad/s):

Caso	(e_{lat} , θ_e)	ω base	ω otim.	Coerente?
Nulo	(0, 0)	0,000	0,000	✓ equilíbrio
Lateral alto dir.	(+5, 0)	+17,279	+17,032	✓ vira p/ pista
Lateral alto esq.	(−5, 0)	−17,279	−17,032	✓ antissimétrico
Angular alto	(0, +2)	−17,279	−17,271	✓ corrige rumo
Aproximação	(+5, +2)	0,000	0,000	✓ não interfere
Conflito crítico	(−5, +2)	−17,279	−17,279	✓ ação máxima

7.2 Rastreamento completo e efeito da otimização

Pista	RMSE Baseline (m)	RMSE Otimizado (m)	Δ
M	0,802	0,705	−12%
A	2,231	2,044	−8%
S	1,647	0,947	−42%
Média	1,560	1,232	−21,0%

As 5 execuções do AG convergiram para 1,23–1,32 m (`resultados/ag_convergence.png`), indicando robustez à semente. Trajetórias em `resultados/baseline_trajectories.png` e `resultados/optimized_trajectories.png` ; superfície de controle em `resultados/superficie_controle.png` .

7.3 Comparação com o artigo de referência

Mancilla et al. reportam que metaheurísticas populacionais reduzem consistentemente o RMSE de rastreamento em relação a controladores não otimizados, com erros da ordem de poucos decímetros a poucos metros conforme a pista. Nossa reprodução, com orçamento de busca reduzido ($20 \times 10 \times 5$ avaliações), replica a **tendência qualitativa** (melhoria consistente, maior nas pistas com curvas mais severas) e a **ordem de grandeza** dos erros, sem pretender igualar os valores absolutos do artigo — comparação direta seria descabida dadas as diferenças declaradas de implementação e orçamento (Seção 6 de `docs/trabalhos_relacionados.md`).

8. Discussão e Análise Crítica

Onde funciona bem. Retas e curvas suaves (erro < 1 m), captura de pista sem oscilação sustentada, equilíbrio exato no caso nulo e antissimetria perfeita (C2/C3) — coerentes com a estrutura antissimétrica da base de regras.

Onde falha ou degrada. Curvas fechadas com velocidade fixa: sem termo de antecipação de curvatura, o robô faz laços de recaptura (pista A, RMSE 2,0 m). O controlador é puramente reativo: desvios são corrigidos apenas depois de ocorrerem.

Parâmetros sensíveis. (i) A largura da zona morta lateral (genes `f` , `i` , `j`): estreita demais causa chattering; larga demais, erro de regime. (ii) O ombro `b` dos termos AN/AP angulares controla quão cedo a correção satura — o AG o reduziu ($1,25 \rightarrow 0,79$), tornando a resposta angular mais agressiva, e compensou alargando a zona morta angular (`a` : $0,5 \rightarrow 0,83$). (iii) A penalidade de não-chegada domina a paisagem de fitness: sem ela, o AG encontra mínimos locais que “cortam caminho”.

Defeito encontrado e corrigido (honestidade metodológica). Na primeira versão, a matriz de regras estava **espelhada no eixo do erro lateral** em relação à convenção de sinal do simulador (`e_lat > 0` = direita): a realimentação lateral ficava positiva e o robô divergia da pista (RMSE ≈ 2085 m, com penalidade), embora a correção angular funcionasse — o que mascarava o defeito nos primeiros metros de simulação. Havia ainda um segundo problema: sem critério de chegada, o robô completava a pista (~ 40 m) em ~ 14 s e continuava vagando até `T_MAX = 50` s, disparando a penalidade de posição final mesmo com bom rastreamento. As correções (espelhar as colunas da matriz e detectar a chegada) elevaram a melhoria do AG de 0,5% para 21,0%. Lição: em sistemas fuzzy, **convenções de sinal são parte da modelagem** e a validação por simulação de malha fechada é indispensável — cenários pontuais isolados não revelavam o problema.

Mamdani $\times$ TSK. A comparação foi feita **experimentalmente** (ver seção “Equipe de 5 integrantes”): um TSK de ordem zero com os mesmos antecedentes e regras superou o Mamdani com

parâmetros baseline ($1,43 \times 1,56$ m — saída mais suave e contínua), mas falhou na pista A ao receber os parâmetros otimizados *para o Mamdani* — evidência de co-adaptação entre parâmetros e motor de inferência. Um TSK com consequentes afins ($\omega = a_0 + a_1 \cdot e_lat + a_2 \cdot \theta_e$) e otimização própria seria ainda mais suave e fácil de ajustar; em troca, perde-se a leitura linguística da saída (“guinada Alta Positiva”) que facilita a auditoria da base de regras. Para um trabalho de reprodução interpretável, Mamdani foi a escolha adequada.

Melhorias futuras. Terceira entrada com curvatura local da pista (antecipação); velocidade variável comandada por segundo sistema fuzzy; otimização também das MFs de saída; comparação experimental Mamdani $\times$ TSK; PSO/DE com o mesmo orçamento.

9. Conclusão

A reprodução mínima viável do controlador de Mancilla et al. (2022) foi implementada, validada e otimizada com sucesso: o sistema Mamdani de 25 regras completa as três pistas de referência com RMSE médio de 1,56 m sem otimização, e o AG real-coded reduz o erro para 1,23 m (−21,0%) ajustando apenas as funções de pertinência das entradas. O trabalho demonstra o ciclo completo exigido pela disciplina — modelagem fuzzy explícita e justificada, implementação reprodutível, validação experimental com análise crítica — e ilustra, com um caso real documentado, por que validação de malha fechada e convenções de sinal explícitas são essenciais em controle fuzzy.

Equipe de 5 integrantes: trilha de ampliação e contribuições

Trilha escolhida: Comparação de modelos — Mamdani $\times$ TSK (experimental). Conforme a Seção 6 da lauda, a equipe de 5 integrantes implementou uma segunda versão do controlador com inferência **Takagi–Sugeno–Kang de ordem zero** (`comparacao_mamdani_tsk.py`): mesmos antecedentes e mesma base de 25 regras, mas consequentes **constantes** (o centroide de cada conjunto de saída: $-17,28 / -0,5 / 0 / +0,5 / +17,28$ rad/s) e saída por **média ponderada** $\omega = \Sigma w_r \cdot c_r / \Sigma w_r$, com peso $w_r = \min(\mu_ \theta, \mu_ e)$ por regra. Resultados (`resultados/comparacao_mamdani_tsk.png`):

Parâmetros das MFs	RMSE médio Mamdani (m)	RMSE médio TSK (m)	Leitura
Baseline (genes = 0,5)	1,5601	1,4335	TSK melhor nas 3 pistas: a média ponderada gera saída mais suave e contínua que o centroide do conjunto agregado
Otimizados pelo AG (para o Mamdani)	1,2340*	668,16 (falha o critério de chegada na pista A: $2000 + 2,06$ de RMSE bruto)	Os parâmetros evoluídos são específicos do motor de inferência : transplantados para o TSK, mantêm erro de rastreo similar mas o robô laça no fim da pista A e não conclui

* 1,234 com os genes publicados (3 casas decimais); 1,2319 com precisão completa (Seção 7.2).

Discussão das diferenças de saída: no caso de erro lateral puro (± 5 m, alinhado), o TSK comanda $\omega = \pm 12,85$ contra $\pm 16,80$ do Mamdani — saída menos saturada, pois a média ponderada dilui o consequente extremo com os termos vizinhos, enquanto no Mamdani o centroide do conjunto agregado é dominado pelo trapézio AN/AP. Isso explica os dois lados da tabela: a suavidade favorece o TSK com parâmetros genéricos (baseline), mas o AG, ao otimizar **para o Mamdani**, explorou exatamente o perfil de saturação desse motor — co-adaptação que não se transfere. Conclusão da trilha: a escolha do mecanismo de inferência e o ajuste de parâmetros **não são decisões independentes**; otimizar o TSK diretamente (consequentes incluídos) é a extensão natural (Seção 8).

A comparação **baseline** × **otimizado** do controlador Mamdani (Seções 7.1–7.2, $-21,0\%$ de RMSE) complementa a trilha como segunda comparação de versões — duas configurações de funções de pertinência com diferenças de saída discutidas.

Contribuições dos integrantes (todos revisaram o código e os documentos e estão preparados para a arguição):

Integrante	Contribuição principal
Brenda Nascimento	Modelagem fuzzy (variáveis, universos, MFs e base de regras) e gestão do repositório
Cauê Jadão	Implementação do motor Mamdani e do simulador; depuração e validação das correções
Augusto Pereira	Algoritmo genético (operadores, fitness) e experimentos de otimização
Fernando Mourão	Cenários de teste, validação experimental e geração de gráficos/evidências
César Ribeiro	Documentação técnica (artigo, manuais) e slides de apresentação

Declaração de Uso de IA

A equipe utilizou IA generativa/agêntica (Claude Code, Anthropic) para esboço de código, revisão/depuração (que identificou os dois defeitos discutidos na Seção 8, posteriormente validados por simulação pela equipe) e rascunho de documentação. Todo material foi revisado, executado e validado pelos integrantes. Declaração completa, com prompts resumidos e revisão crítica item a item, em `docs/declaracao_uso_ia.md`.

Referências

[1] MANCILLA, A.; GARCÍA-VALDEZ, M.; CASTILLO, O.; MERELO-GUERVÓS, J. J. Optimal Fuzzy Controller Design for Autonomous Robot Path Tracking Using Population-Based Metaheuristics. *Symmetry*, v. 14, n. 2, art. 202, 2022. DOI: 10.3390/sym14020202.

[2] ANTONELLI, G.; CHIAVERINI, S.; FUSCO, G. A fuzzy-logic-based approach for mobile robot path tracking. *IEEE Transactions on Fuzzy Systems*, v. 15, n. 2, p. 211–221, 2007.

[3] MAMDANI, E. H.; ASSILIAN, S. An experiment in linguistic synthesis with a fuzzy logic controller. *International Journal of Man-Machine Studies*, v. 7, n. 1, p. 1–13, 1975.

[4] TAKAGI, T.; SUGENO, M. Fuzzy identification of systems and its applications to modeling and control. *IEEE Transactions on Systems, Man, and Cybernetics*, v. SMC-15, n. 1, p. 116–132, 1985.

[5] CORDÓN, O.; HERRERA, F.; HOFFMANN, F.; MAGDALENA, L. *Genetic Fuzzy Systems: Evolutionary Tuning and Learning of Fuzzy Knowledge Bases*. Singapore: World Scientific, 2001.

[6] PRECUP, R.-E.; HELLENDORRN, H. A survey on industrial applications of fuzzy control. *Computers in Industry*, v. 62, n. 3, p. 213–226, 2011.

[7] ZADEH, L. A. Fuzzy sets. *Information and Control*, v. 8, n. 3, p. 338–353, 1965.

Ferramentas: Python; NumPy; SciPy; Matplotlib; scikit-fuzzy (documentação oficial); Jupyter; Git/GitHub.